

(https:// /  Q...



HackMD

(https://hackmd.io?utm_source=view-page&utm_medium=logo-nav)

Quiz1 of Computer Architecture (2022 Fall)

Solutions

Problem A

Popular programming journal *Obscure C Techniques for Experts* has published a novel way to save space for a doubly-linked list program. Instead of storing two pointers (one next and one previous), this new technique stores a single value: the XOR of previous and next pointers. Check Wikipedia [XOR linked list](https://en.wikipedia.org/wiki/XOR_linked_list) (https://en.wikipedia.org/wiki/XOR_linked_list) for details.

The below code (filename `xorlist.c`) implements [XOR linked list](https://en.wikipedia.org/wiki/XOR_linked_list) (https://en.wikipedia.org/wiki/XOR_linked_list):

```

#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

#define LIST_START 0
#define LIST_END 5

typedef struct {
    int data;
    intptr_t link;
} xorlist_t;

void dump_list(xorlist_t *pt)
{
    intptr_t prev = (intptr_t) NULL;

    while (pt) {
        printf("%d\n", pt->data);
        xorlist_t *current = pt;
        /* Decode xored pointers with last node's address to find next */
        pt = (xorlist_t *) (pt->link ^ prev);
        prev = (intptr_t) current;
    }
}

/* insert head */
void insert_head(xorlist_t **head, int data)
{
    xorlist_t *new_node = malloc(sizeof(xorlist_t));
    new_node->data = data;

    if (!*head) {
        new_node->link = (intptr_t) NULL;
    } else {
        /* Update original link of head node */
        (*head)->link = A01 /* Fill Your Code */ ^ (intptr_t) new_node;
        new_node->link = A02 /* Fill Your Code */;
    }

    *head = new_node;
}

/* remove a node from head */
void remove_head(xorlist_t **head)
{
    if (!(*head)) return;

    xorlist_t *tmp = (xorlist_t *) (*head)->link;
    /* Update the link of new head */
    if (tmp)
        tmp->link ^= A03 /* Fill Your Code */;
    free(*head);
    *head = tmp;
}

void release_list(xorlist_t *pt)

```

```

{
    intptr_t prev = (intptr_t) NULL;
    while (pt) {
        xorlist_t *current = pt;
        pt = (xorlist_t *) (pt->link ^ prev);
        prev = A04 /* Fill Your Code */;
        free(current);
    }
}

int main()
{
    xorlist_t *head = malloc(sizeof(xorlist_t)), *tail;
    xorlist_t *pt = head;
    intptr_t last_node = (intptr_t) NULL;
    for (int c = LIST_START; c < LIST_END; ++c) {
        xorlist_t *new_node = malloc(sizeof(xorlist_t));
        *pt = (xorlist_t){.data = c, .link = (intptr_t) new_node ^ last_node};
        last_node = (intptr_t) pt;
        pt = new_node;
    }
    *pt = (xorlist_t){.data = LIST_END, .link = last_node ^ (intptr_t) NULL};
    tail = pt;

    insert_head(&head, 99);
    dump_list(head);

    remove_head(&tail);
    dump_list(tail);

    release_list(head);

    return 0;
}

```

The corresponding program output:

```

99
0
1
2
3
4
5
4
3
2
1
0
99

```

Ensure your code meets the expected operations and initialization of such a list as well as the insertion, and deletion, of elements from such a list.

Obviously, the above C code listing was incomplete, you shall provide the functioned implementations. A01 , A02 , A03 , and A04 are C expressions. You must obey the following rules when filling them:

- Write shorter code as possible as you can.
- Do necessary casting to eliminate compilation warnings.
- The `intptr_t` and `uintptr_t` types are extremely useful for casting pointers when you want to do address arithmetic. You shall consider to explicitly use the types provided by `<inttypes.h>` .
- Follow the consistent coding style. That is, we prefer `(xorlist_t *) (*head)->link` to `(xorlist_t*)(*head)->link` or `(xorlist_t *)(*head)->link` . Be aware of the **spaces!** Details determine success or failure.

- A01 = ?
`(*head)->link`
- A02 = ?
`(intptr_t) *head`
- A03 = ?
`(intptr_t) *head`
- A04 = ?
`(intptr_t) current`

Problem B

Fill in B01 , B02 , B03 and B04 to complete the `reverse` function which takes in a `head` to the head of a linked list and returns a new copy of the linked list in reverse order. You must allocate space for the new linked list that you return. An (incomplete) example program using `reverse` is also shown below.

```

#include <assert.h>
#include <stdlib.h>

struct list_node {
    int val;
    struct list_node *next;
};

struct list_node *reverse(B01 /* Fill Your Code */)
{
    struct list_node *next = NULL, *ret;
    while (*head) {
        ret = malloc(sizeof(struct list_node));
        ret->val = B02 /* Fill Your Code */;
        ret->next = B03 /* Fill Your Code */;
        next = B04 /* Fill Your Code */;
        *head = (*head)->next;
    }
    return ret;
}

/* Assume that NEW_LL_1234() properly malloc's a linked list
 * 1 -> 2 -> 3 -> 4, and returns a pointer that points to the first
 * list_node in the linked list. Assume that before test_reverse
 * returns, head and ret will be properly freed.
 */
void test_reverse()
{
    struct list_node *head = NEW_LL_1234();
    assert(head->val == 1); /* returns True */
    assert(head->next->val == 2); /* returns True */
    struct list_node *ret = reverse(&head);
    assert(head != ret); /* ret is a new copy of the original list */
    assert(ret->val == 4); /* should return True */
}

```

You shall provide the functioned implementations. B01 is a declaration for function parameter. B02 , B03 , and B04 are C expressions. You must obey the following rules when filling them:

- Write shorter code as possible as you can.
- Do necessary casting to eliminate compilation warnings.
- Follow the consistent coding style. That is, we prefer `struct list_node *h` to `struct list_node* h`. Be aware of the **spaces**! Details determine success or failure.

- B01 = ?
 struct list_node **head
- B02 = ?
 (*head)->val
- B03 = ?
 next
- B04 = ?
 ret

Problem C

Let's try to determine endianness (<https://en.wikipedia.org/wiki/Endianness>), at compile time. If someone needs to know the endianness at compile time, there are 2 different use cases:

- in some rare cases one needs to know the endianness to lay out structs and the like, for example an RGB (https://en.wikipedia.org/wiki/RGB_color_spaces) union; or directly in the code.
- in most cases, just a conversion from one endian to the other is needed, e.g. when a protocol defines that a value is stored in a specific endianness.

In the majority of use cases, one wants just to convert from little to big endian, and vice versa (or actually to and from host endianness!). Assume that we are going to provide endian conversion functions/macros start with the `end_` prefix.


```

/* Return a value in which the order of the bytes in 4-byte arguments is reversed. */
static inline uint32_t end_bswap32(uint32_t __x)
{
    return (__x >> 24) | (__x >> 8 & C01) | (__x << 8 & C02) |
        (__x << 24);
}

/* Host to Big Endian 32-bit */
static inline uint32_t end_htobe32(uint32_t n)
{
    union {
        int i;
        char c;
    } u = {1};
    return u.c ? C03 : C04;
}

/* Host to Little Endian 32-bit */
static inline uint32_t end_htole32(uint32_t n)
{
    union {
        int i;
        char c;
    } u = {1};
    return u.c ? C05 : C06;
}

```

You shall provide the functioned implementations. Both `C01` and `C02` are hexadecimal integer literals, meaning that they should start with `0x`. `C03`, `C04`, `C05`, and `C06` are C expressions. You might consider to call `end_bswap32` function when it is necessary. You must obey the following rules when filling them:

- Write shorter code as possible as you can.
- Do necessary casting to eliminate compilation warnings.
- Follow the consistent coding style. That is, we prefer `end_bswap32(n)` to `end_bswap32( n )`. Be aware of the **spaces**! Details determine success or failure.

- C01 = ?
0xff00
- C02 = ?
0xff0000
- C03 = ?
end_bswap32(n)
- C04 = ?
n
- C05 = ?
n
- C06 = ?
end_bswap32(n)

Problem D

Given the following bitstream (<https://en.wikipedia.org/wiki/Bitstream>). 11111100₂, answer the following questions:

1. What is the value if it was interpreted in two's complement? __ D01 __ (Answer in decimal)

- D01 = ?
-4
(Flip all the bits and add 1) * -1 = (0b0000 0011 + 1) * -1 = -4

2. You are given the following field breakdown and specifications of an 8-bit floating point, which follows the same rules as standard 32-bit IEEE floats, except with different field lengths: (Sign: 1 bit; Exponent: 3 bits; Significand: 4 bits)

Exponent Value	Significand Value	Floating Point Value
Smallest	Zero, Non-Zero	±0, Denormalized
Largest	Zero, Non-zero	±Infinity, NaN

What is the floating point value of 11111100₂? __ D02 __

- D02 = ?

NaN

Exponent field is the largest exponent (0b111) and the significant is non-zero → NaN

- Now we modify the floating point description in part 2, so that the exponent field is now in two's complement instead of in bias notation. Compute the floating point value of 11111100_2 . __ D03 __

- D03 = ?

-0.875

Exponent: 0b111 = -1, Signif: 0.75 = $(-1) \times 2^{-1} \times 1.75 = -0.875$

Problem E

You received a sequence of IEEE standard 16-bit floating point numbers from a trusted source. Note that a 16-bit floating point is 1 sign bit, 5 exponent bits, and 10 mantissa bits. The bias for the exponent is -15. Unfortunately, some of the data was corrupted during communication, rendering it unreadable. For the following problems, we will use x to refer to a bit that was corrupted (in other words, we don't know what the sender wanted that bit to be). For example, if I received the data $0b0xx1$, the sender sent one of $0b0001$, $0b0101$, $0b0011$, or $0b0111$.

- You receive the data $0b1110xxxxxxxxxxxx$. What is the decimal value of the smallest number the sender could have sent (i.e. it is less than all of the other possibilities)? __ E01 __ You must provide the decimal form, do not leave as a power of 2.

- E01 = ?

-8188

By the previous observation, the smallest number is encoded by 0xEFFF.

This has sign bit 1, exponent 0b11011 - 15 = 12, and mantissa (1).1111111111

$= 2 - 2^{-10}$. The answer would be

$$-4096 \times (2 - 2^{-10}) = -8192 + 4 = -8188$$

- You receive the data "0b0x1x0x1x0x1x0x1x". What is the hexadecimal encoding of the biggest number the sender could have sent? __ E02 __

- E02 = ?

0x7777

One property of floating point numbers is that their order is the same as that of sign-magnitude integers (ignoring NaNs); for example, 0x5555 < 0x7555. In order to maximize the number, we therefore want to set all the x s to 1. This yields the encoding 0x7777.

Problem F

Following the bit-level floating-point coding rules, implement the function with the following prototype:

```
/*
 * Compute (int) f
 * If conversion causes overflow or f is NaN, return 0x80000000
 */
typedef unsigned float_bits;
int float_to_int(float_bits f);
```

For IEEE 754 single-precision floating-point number f , this function computes $(\text{int}) f$. Your function should round toward zero. If f cannot be represented as an integer, then the function should return `0x80000000`.

```

static inline float u2f(unsigned u)
{
    return *(float *) &u;
}

int float_to_int(float_bits f)
{
    const unsigned sign = f >> 31;
    const unsigned exp = f >> F01 /* Fill Your Code */ & 0xFF;
    const unsigned frac = f & F02 /* Fill Your Code */;
    const unsigned bias = 0x7F;
    int result;
    if (exp < bias) {
        /* the float number is less than 1 */
        result = 0;
    } else if (exp >= 31 + bias) {
        /* overflow */
        result = 0x80000000;
    } else {
        /* normal */
        unsigned E = exp - bias;
        unsigned M = frac | 0x800000;
        if (E > F03 /* Fill Your Code */) {
            result = F04 /* Fill Your Code */;
        } else {
            /* round to zero */
            result = F05 /* Fill Your Code */;
        }
    }
    return sign ? -result : result;
}

```

Fill F01, F02, F03, F04 and F05 to make the above C code really functioned as expected. Both F01 and F03 are decimal. F02 is a hexadecimal integer literal. Both F04 and F05 are C expressions. You must obey the following rules when filling them:

- Write shorter code as possible as you can.
- Follow the consistent coding style. That is, we prefer `M << (E - 1)` to `M<<(E-1)`. Be aware of the **spaces**! Details determine success or failure.

- F01 = ? (Answer in decimal)
23
- F02 = ? (Answer in HEX)
0x7FFFFFFF
- F03 = ? (Answer in decimal)
23
- F04 = ?
 $M \ll (E - 23)$
- F05 = ?
 $M \gg (23 - E)$

Problem G

Implement C function which gets a number and returns if it is positive (i.e., > 0).

```
if <= 0 then return false, if > 0 then return true.
```

Example: $0 \rightarrow \text{false}$, $-12 \rightarrow \text{false}$, $29 \rightarrow \text{true}$.

However, you CAN NOT use any flow control (e.g. if, else, for, switch, case, goto, while, ?) nor logical operators (e.g. $\&\&$, $\|\|$, $>$, $<$, $==$, $<=$, $>=$). Instead, you SHALL use operator $\ll$, $\gg$, $\&$, $|$, $+$, $\wedge$, $\sim$, $!$ to implement such routine. Assume `sizeof(int) = 4`.

```

int is_positive(int x)
{
    const int mask = G01 /* Fill Your Code */;
    /* place x's MSB in the least bit
    * if x negative, tmp is 11111111
    * if positive/0, tmp is 00000000
    */
    int tmp = x >> 31;

    /* keep just the least bit by AND to mask(00000001)
    * if x negative, tmp is 00000001
    * if positive/0, tmp is 00000000
    */
    tmp &= mask;

    /* if x is 0 -- now it is 00000001, else it is 0 */
    x = !x;

    /* if x is negative or '0', make x to 1
    * if x is positive, make x to 0
    */
    G02 /* Fill Your Code */;

    /* if x positive, make it 1.
    * if negative/0, make it 0.
    */
    x = !x;

    /* return 1 if x > 0, else return 0 */
    return x;
}

```

Fill G01 and G02 to make the above C code really functioned as expected. G01 is decimal, and G02 is a C expression. You must obey the following rules when filling them:

- Write shorter code as possible as you can.
- Follow the consistent coding style. That is, we prefer `x &= 1` to `x = x&1`. Be aware of the **spaces**! Details determine success or failure.

- G01 = ?

1

- G02 = ?

`x |= tmp OR x = x | tmp OR x = tmp | x OR x ^= tmp OR x = x ^ tmp OR`
`x = tmp ^ x`